

MTech LLM Project Report (Sem-III)
on
LLM: Efficient Fine-tuning under Limited GPU Memory
(Optimize memory-efficient tuning)

Submitted by

Rakesh Kumar - 24MDA066

Under the guidance of

Dr. Krishnendu



INDIAN INSTITUTE OF
INFORMATION
TECHNOLOGY

LLM: Efficient Fine-tuning under Limited GPU Memory

(Optimize memory-efficient tuning)

Phase-1: Efficient Fine-Tuning with Tiny Llama

Phase-2: Comparative Fine-Tuning Across Multiple LLMs.



INDIAN INSTITUTE OF
INFORMATION
TECHNOLOGY

Phase I: Efficient Fine-Tuning with Tiny Llama



INDIAN INSTITUTE OF
INFORMATION
TECHNOLOGY

Large Language Models (LLMs) such as LLaMA, Mistral, and GPT contain billions of parameters. During conventional fine-tuning, all model parameters are updated, requiring storage for model weights, gradients, optimizer states, and intermediate activations. As the model size increases, GPU memory consumption grows significantly, often exceeding the capacity of commonly available GPUs.

For example, fine-tuning a 7-billion parameter model may require more than 40 GB of GPU memory, while larger models require several high-end GPUs. This creates a major barrier for students, researchers, and organizations with limited computational resources.

Parameter-Efficient Fine-Tuning (PEFT) methods such as LoRA and QLoRA address this challenge by reducing the number of trainable parameters and minimizing GPU memory usage.

LoRA freezes the pretrained model and trains only a small set of low-rank adapter matrices, whereas QLoRA further compresses the frozen model using 4-bit quantization. These techniques enable efficient fine-tuning on affordable GPUs without significantly compromising model performance.

Source: <https://huggingface.co/datasets/roneneldan/TinyStories>

Attribute	Details
Full training set	2,119,719 stories
Full validation set	21,990 stories
Vocabulary	Simplified, child-level English
Story generation source	GPT-3.5 / GPT-4 synthetic generation
Subset used (this project)	2,000 train / 500 validation
Max tokenized length	256 / 516 tokens

Why Tiny Stories Dataset?

- 2.1M+ synthetic stories generated by GPT-3.5 and GPT-4.
- Simple vocabulary and short narratives enable fast and stable model learning.
- Computationally efficient, reducing training time and GPU memory requirements.
- Ideal benchmark for comparing LoRA and QLoRA under constrained hardware.
- Clean, coherent, and consistent text suitable for supervised story generation.

Official Hugging Face Repository: <https://huggingface.co/TinyLlama/TinyLlama-1.1B-Chat-v1.0>

Architecture	Llama-style decoder-only transformer
Model size	~1.1B
Model type	Decoder-only causal LM
Architecture	Llama2-style Transformer
Suitability	Designed for restricted compute and memory
Hidden Size	2048 Tokens
Variant	Chat-tuned (v1.0)
Fine-Tuning Support	LoRA, QLoRA, Full Fine-Tuning

Why this model?

- ✓ Fast training: Small model enables shorter training time
- ✓ Memory efficient
- ✓ Comparable with larger LLMs
- ✓ Freely available for research and education

Instead of updating billions of parameters LoRA:

- Freezes the pretrained model weights (W).
- Introduce two small trainable matrices (A and B).
- Learn only the low-rank adaptation during fine-tuning.
- Lowers GPU memory usage and computational cost.
- Preserve the original knowledge while adapting to a new task.

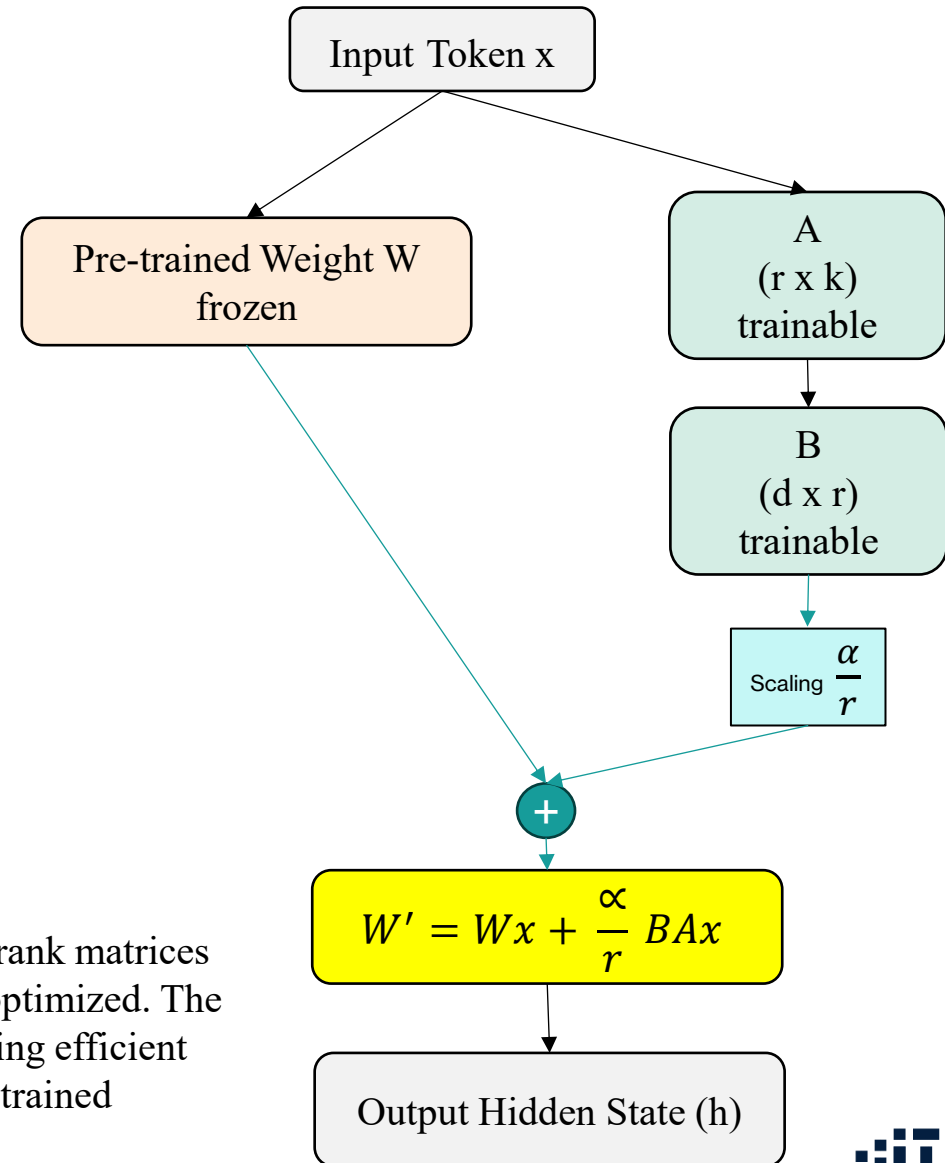
Mathematical Formulation

$$W' = W + \frac{\alpha}{r} BA$$

where

- W → Frozen pretrained weight matrix
- A, B → Trainable low-rank matrices
- r → Rank of adaptation
- α → Scaling factor

LoRA freezes the pretrained weight matrix W and learns two small trainable low-rank matrices A and B . Instead of updating all model parameters, only the adapter matrices are optimized. The scaled adapter output $(\alpha/r)BAx$ is added to the original model output Wx , enabling efficient fine-tuning with significantly fewer trainable parameters while preserving the pretrained knowledge.



Experiment Objectives

- Demonstrate Parameter-Efficient Fine-Tuning (PEFT) using LoRA.
- Fine-tune Tiny Llama by updating only lightweight adapter layers.
- Reduce GPU memory and computational requirements compared with full fine-tuning.
- Generate coherent Tiny Stories-style text after fine-tuning.
- Establish a baseline for comparison with QLoRA.

LoRA Configuration:

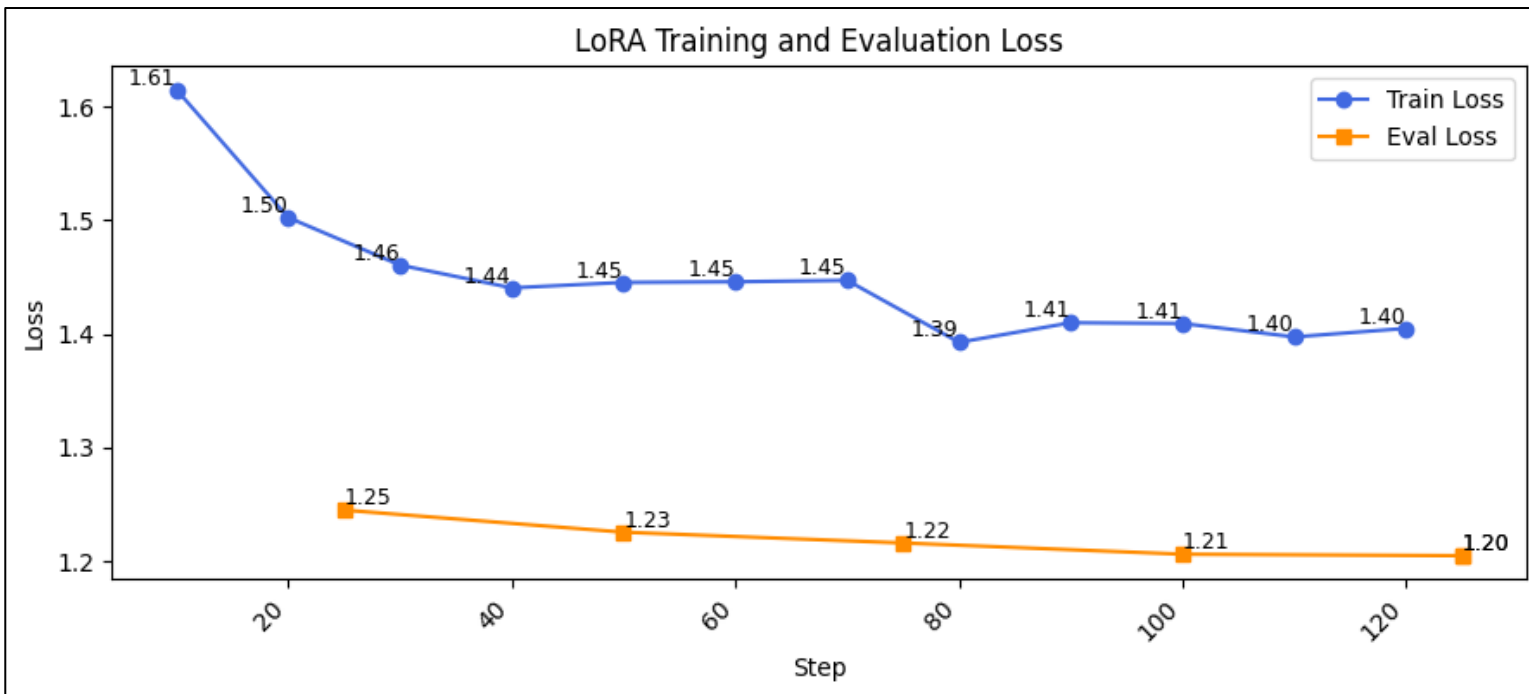
- **Rank ($r = 16$):** Controls the size of the low-rank adapters, balancing model capacity and parameter efficiency.
- **LoRA Alpha ($\alpha = 32$):** Scales the contribution of the LoRA adapters during training.
- **LoRA Dropout (0.05):** Reduces overfitting by randomly dropping adapter
- **Target Modules:** Applied to **q_proj**, **k_proj**, **v_proj**, and **o_proj**, enabling efficient adaptation of the Transformer's attention mechanism.

Training Configuration

- **Learning Rate (2×10^{-4}):** Determines the step size for updating LoRA adapter parameters.
- **Optimizer (paged_adamw_32bit):** Memory-efficient optimizer designed for LoRA/QLoRA training.
- **Batch Size:** 2 samples per iteration.
- **Gradient Accumulation:** 8 steps, giving an **effective batch size of 16**.
- **Mixed Precision (FP16):** Reduces GPU memory usage and accelerates training.

These hyperparameters balance adaptation capacity, regularization, and GPU efficiency for training on a small dataset and limited VRAM.

Performance Metric	LoRA Result
Training Loss	1.4469
Validation Loss	1.2050
Perplexity	3.3367
Training Time	1280.74 sec
Samples/sec	1.562
GPU Memory Allocated (GB)	6.22
GPU Memory Reserved (GB)	6.36



QLoRA (Quantized LoRA) extends LoRA by storing the pretrained model weights in 4-bit precision while keeping the LoRA adapter matrices trainable in higher precision (FP16). This significantly reduces GPU memory usage without updating the original pretrained weights.

- Quantizes the pretrained Tiny Llama weights to 4-bit NF4 representation.
- Keeps the quantized weights frozen during fine-tuning.
- Trains only the LoRA adapter matrices (A and B) in FP16 precision.
- Dequantizes the 4-bit weights on-the-fly during the forward pass.
- Produces outputs using both the quantized base model and LoRA update

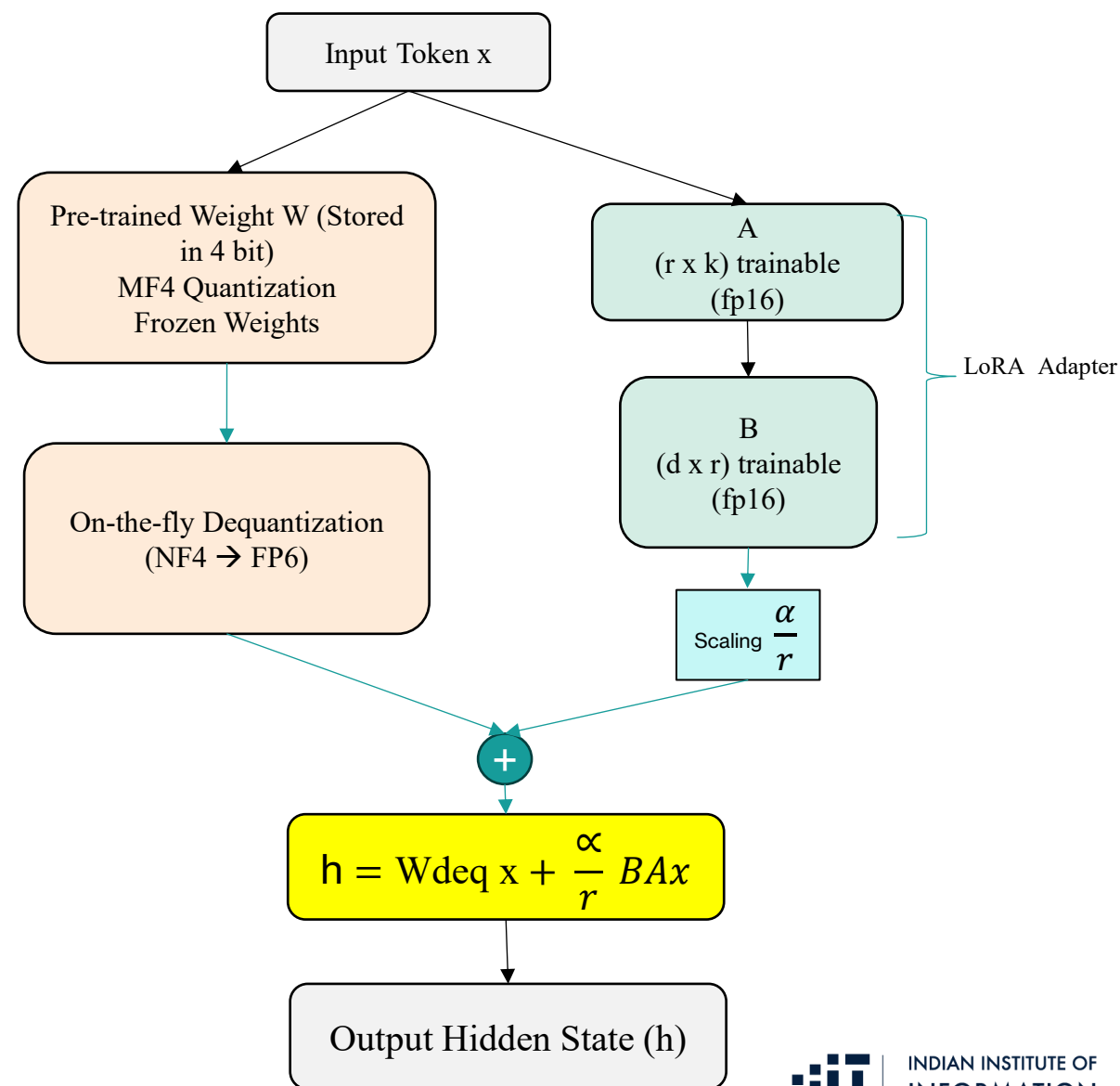
Mathematical Formulation

$$h = W_{\text{deq}}x + \frac{\alpha}{r} B A x$$

where

- W_{deq} : represents the dequantized, weights used during computation

QLoRA compresses the pretrained model into 4-bit NF4 format while training only FP16 LoRA adapters, enabling memory-efficient fine-tuning with minimal impact on model quality.



QLoRA SETUP SUMMARY

Base Model : TinyLlama/TinyLlama-1.1B-Chat-v1.0
Quantization : 4-bit
Quantization Type : nf4
Compute Dtype : float16
Double Quantization : True

Total Parameters : 1,104,553,984
Trainable Parameters : 4,505,600
Trainable Percentage : 0.4079%

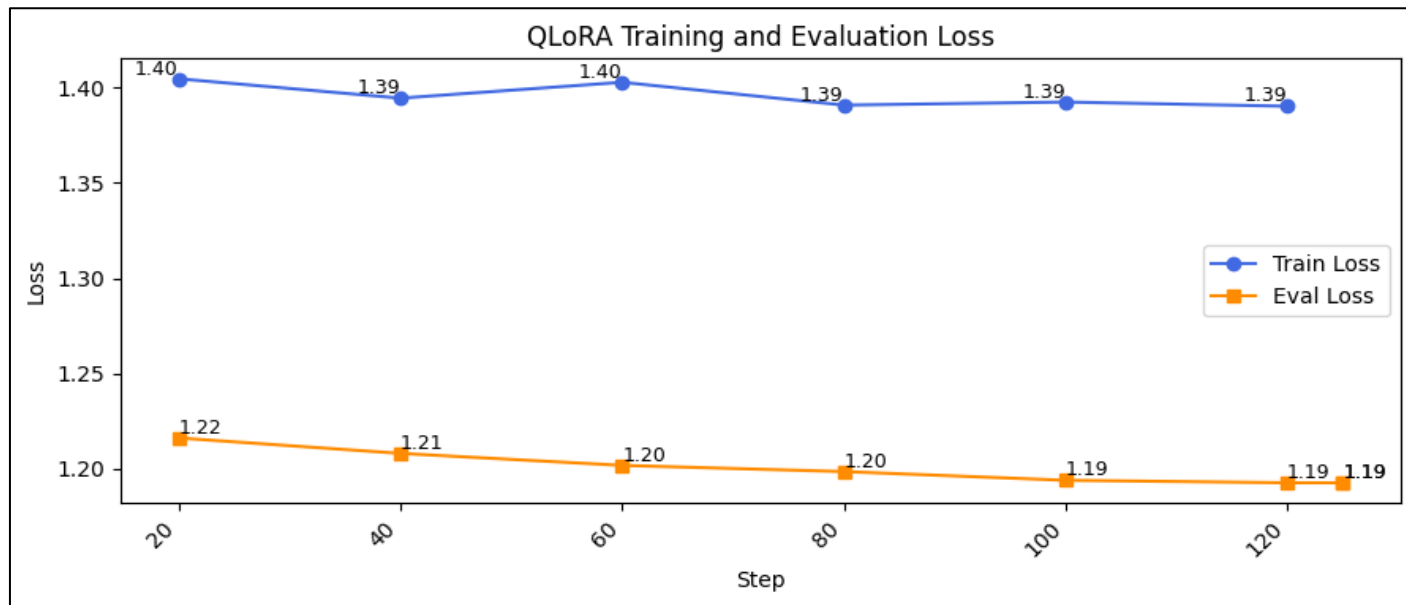
Experiment Objectives

- Demonstrate Quantized Parameter-Efficient Fine-Tuning (QLoRA) using 4-bit NF4 quantization and LoRA adapters.
- Compress the pretrained Tiny Llama model to 4-bit precision while keeping only the LoRA adapters trainable.
- Reduce GPU memory consumption beyond standard LoRA without sacrificing text generation quality.
- Fine-tune Tiny Llama efficiently on a single Tesla T4 (16 GB VRAM) GPU.
- Compare QLoRA against LoRA using identical datasets and training settings.

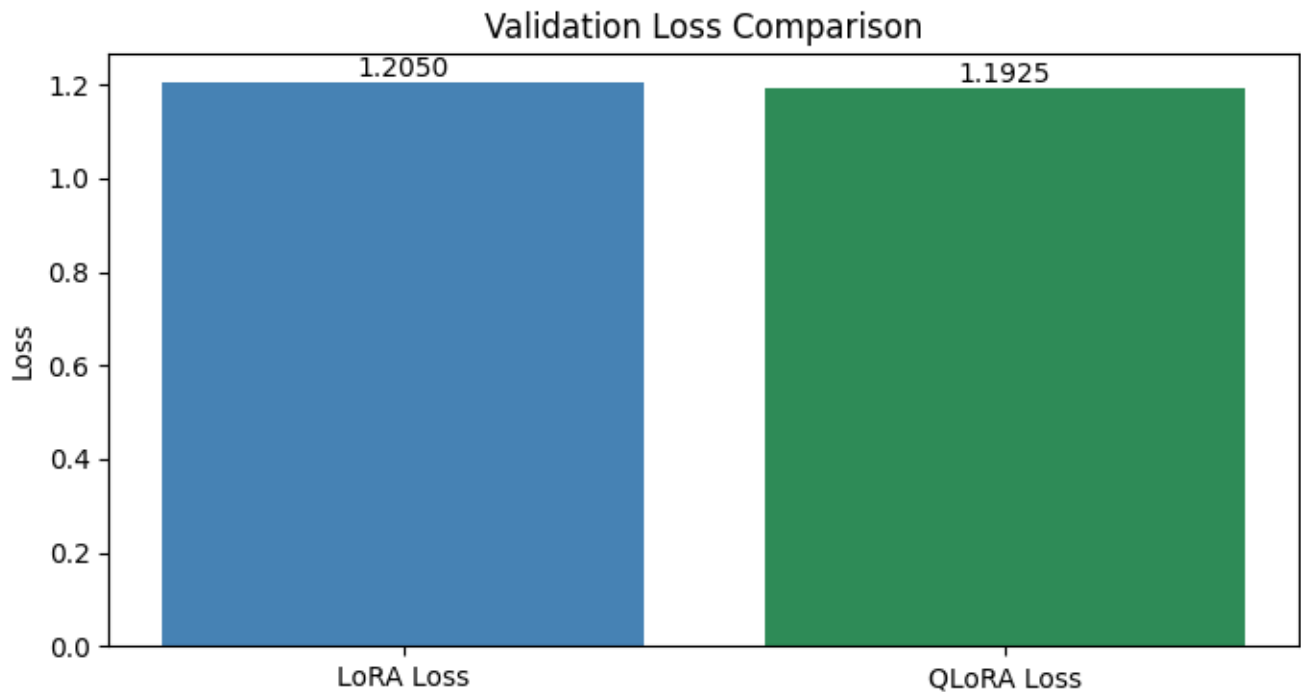
These hyperparameters combine 4-bit NF4 quantization with LoRA adapters to minimize GPU memory usage while preserving efficient fine-tuning and model performance.

LoRA	QLoRA
Frozen base model stored in FP16	Frozen base model stored in 4-bit NF4
LoRA adapters trained in FP16	LoRA adapters trained in FP16
No quantization	4-bit quantization + double quantization
Lower memory than full fine-tuning	Even lower memory than LoRA

Performance Metric	LoRA Result
Training Loss	1.3971
Validation Loss	1.2198
Perplexity	3.3895
Training Time	547.54 sec (9.8 m)
Samples/sec	4.2230
Steps/sec	0.2640
Trainable Parameters	4,505,600 ~ 4.51M (0.4079%)
Peak GPU Memory	3..79 GB



Sl. No.	Metric	LoRA	QLoRA
0	Training Method	LoRA	QLoRA
1	Base Model Precision	Full precision / fp16 depending on setup	4-bit quantized
2	Trainable Parameters	Small adapter set	Small adapter set
3	GPU Memory Use	Moderate	Lower
4	Training Speed	Fast	Efficient under memory limits
5	Validation Loss	1.205	1.1925
6	Perplexity	3.3367	3.2952
7	Best For	Quality-first baseline	Best for limited GPU memory



BASE MODEL OUTPUT:

Once upon a time, there was a little rabbit named Ralph. He lived in a big tree with his family. One day, a little fox came by and asked Ralph to show him the way to the forest where the gold was buried. "I don't know," said Ralph. "But I will try my best." Ralph went through the forest, searching for clues. He found an old map in a tree and read it carefully. The map led him to the Gold Mine, but he didn't see any sign of the gold. It seemed like the gold had vanished from the world forever. One day, Ralph met a wise old owl who told him that there was a chance that the gold would return one day. Ralph was not convinced but agreed to help the owl find the secret entrance to the mine. The owl set off on a journey, using his sharp eyesight and keen sense of smell to track down the mine entrance. After many days, the owl finally discovered the entrance under the mountain. When they reached the mine, Ralph and the owl spent hours digging and searching. Finally, after much hard work, they found a small treasure chest full of gold coins.

=====

LORA MODEL OUTPUT:

Once upon a time, there was a little rabbit named Bob. He was very small and lived in a nice place near the forest. One day, he saw something strange. It was a big snake! Bob was scared. He ran away quickly but he was too late. The snake had already bitten his tail and started to move forward. Bob tried to run faster but the snake was so strong that it could not stop. In the end, Bob was bitten by the snake. He cried out loud and ran away as fast as he could. But as soon as he was far away, the snake grabbed him back and twisted his body until it broke. The snake was happy because now it had a new friend. Bob had lost his tail and couldn't run anymore. The snake was proud of how it had bitten and killed its prey.

=====

QLORA MODEL OUTPUT:

Once upon a time, there was a little rabbit named Mia. Mia lived in a dark, damp cave with her momma and her 3 siblings. One day, Mia's momma told her to go outside so she could see the world. Mia went outside and saw that it was very cold! She froze, so she ran back inside her cave. When Mia came home, her momma said, "Mia, you have to stay inside and get warm until the weather gets better." Mia felt sad because she wanted to go outside and play with her friends, but she knew it was best not to do so. The next day, when Mia went out, she saw a group of animals going around and exploring the world. They all looked like fun, but they were also so cold. Mia knew she had to be brave and try something new - so she decided to take a few steps outside. To her surprise, she found out that it was so much warmer outside than it was inside. It was like being in a paradise! Mia had never seen anything like this before and she was amazed by how different things looked outside compared to what they had seen inside their cave

Phase - II: Comparative Fine-Tuning Across Multiple LLMs



INDIAN INSTITUTE OF
INFORMATION
TECHNOLOGY

Compare LoRA vs QLoRA parameter-efficient fine-tuning on Tiny Stories across three models to show memory savings without sacrificing story quality, on a Kaggle T4 GPU

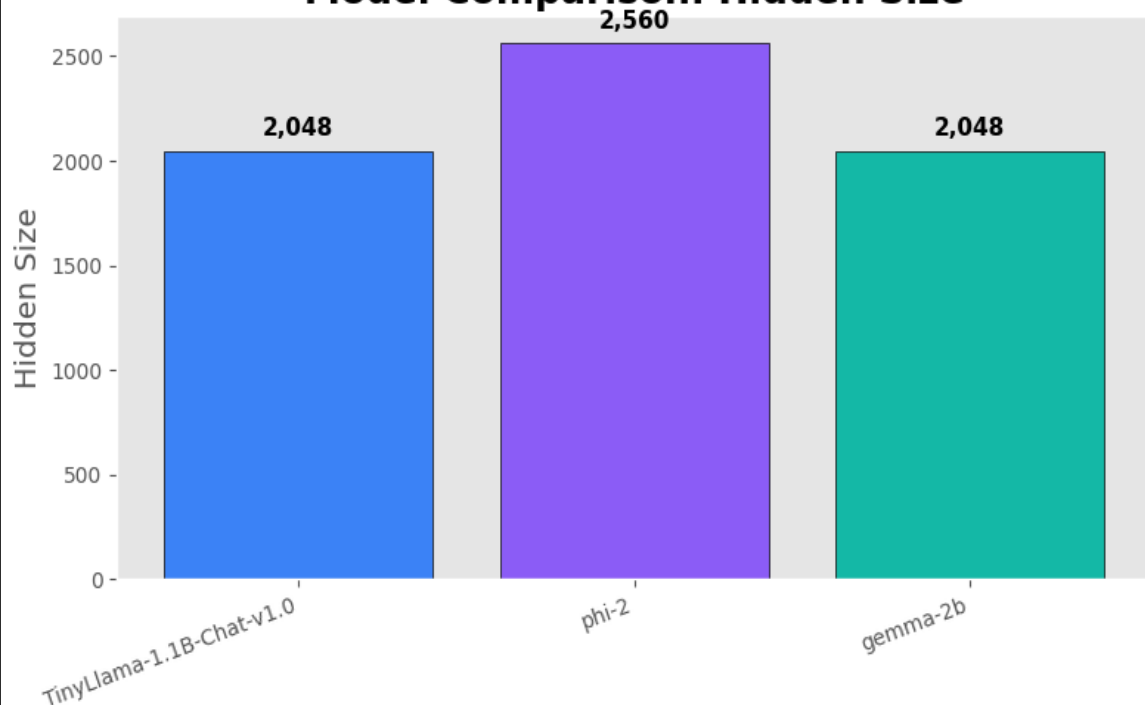
Models tested:

- TinyLlama-1.1B-Chat-v1.0,
- Microsoft Phi-2 (2.79B),
- Google Gemma-2B

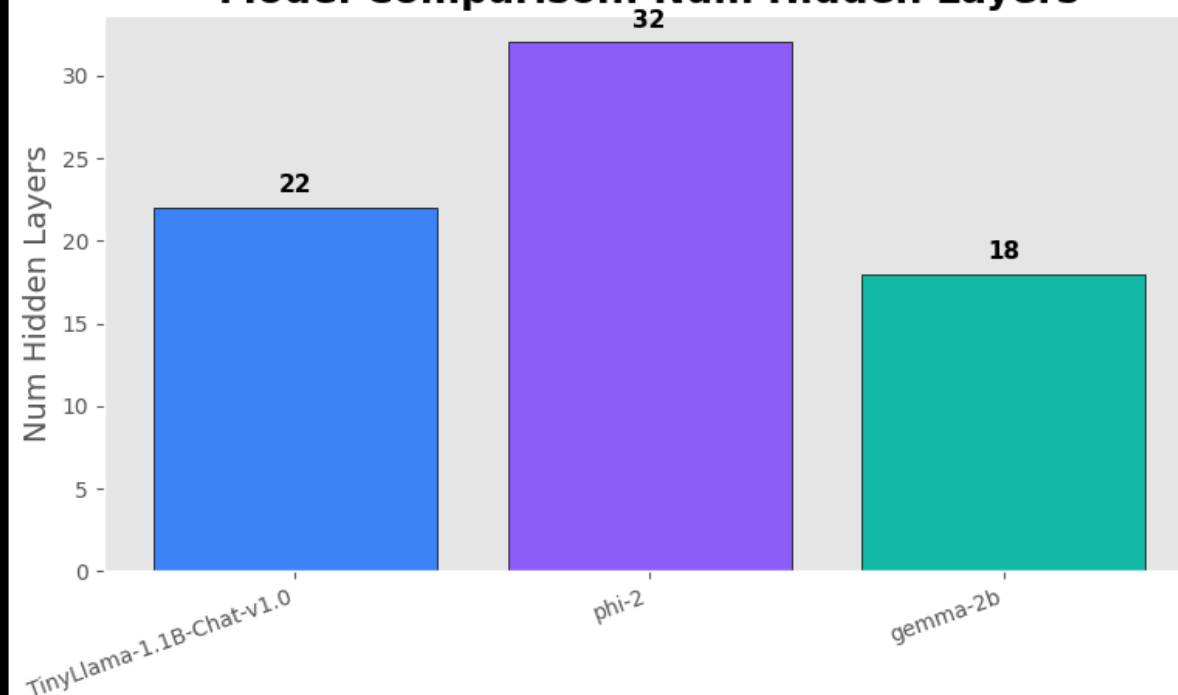


each with both LoRA and QLoRA (6 runs total).

Model Comparison: Hidden Size



Model Comparison: Num Hidden Layers

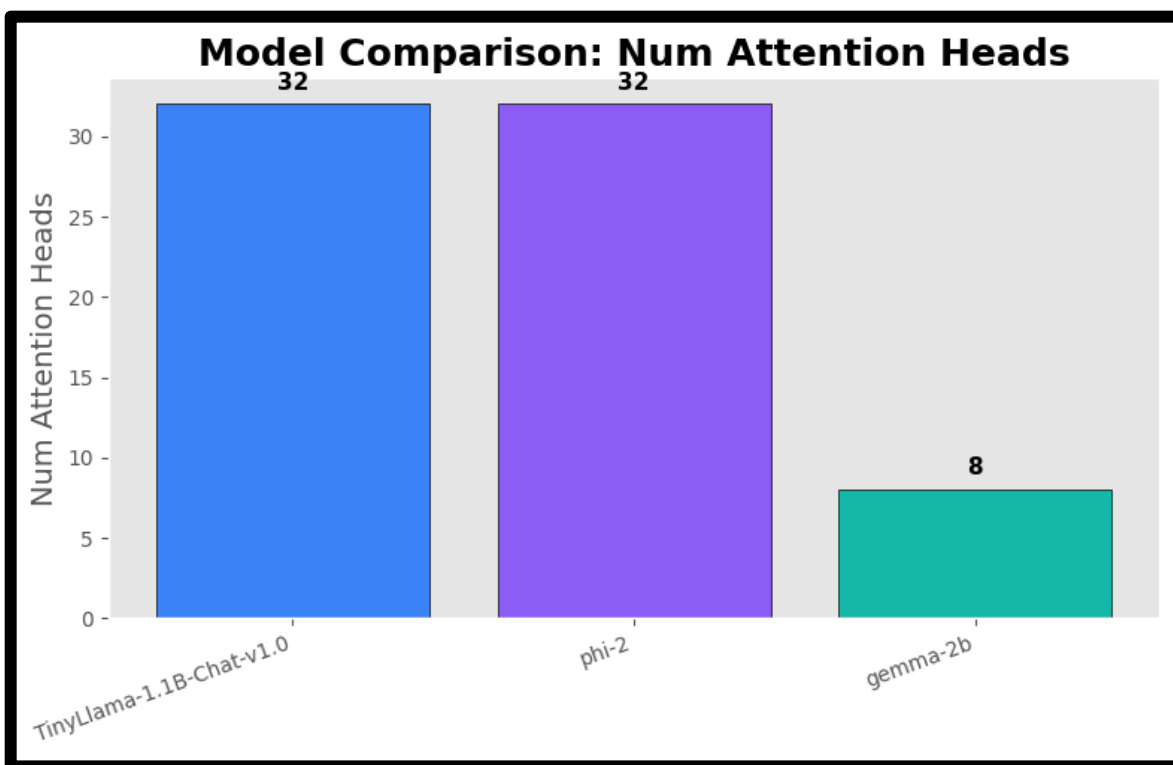


Hidden Size

- **Phi-2: 2,560 vs 2,048** for Tiny Llama /Gemma-2B → wider token representations and higher model capacity.
- **Experiment impact:** Greater width increases computation and memory demand, contributing to Phi-2's higher resource requirements.

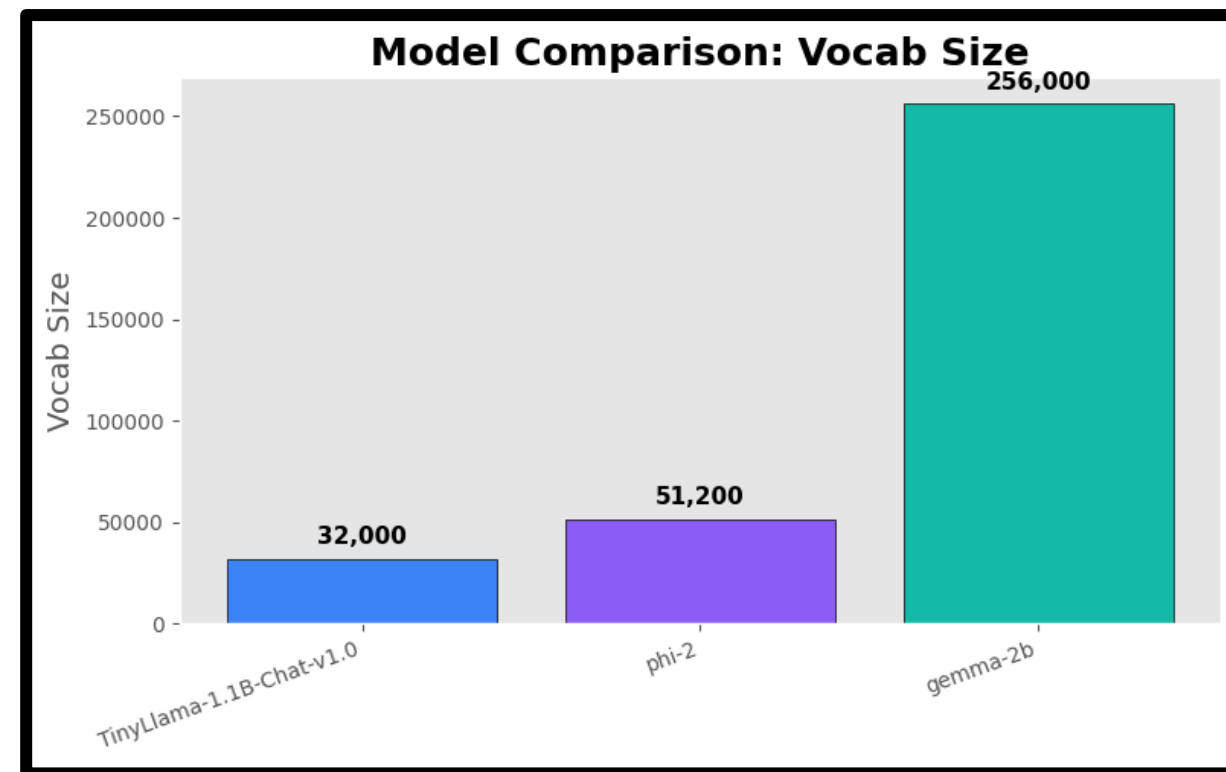
Hidden Layers:

- **Phi-2: 32 layers vs 22/18** → deepest architecture with more sequential representation transformations.
- **Experiment impact:** Greater depth increases computational workload and contributes to longer fine-tuning time.



Attention Heads

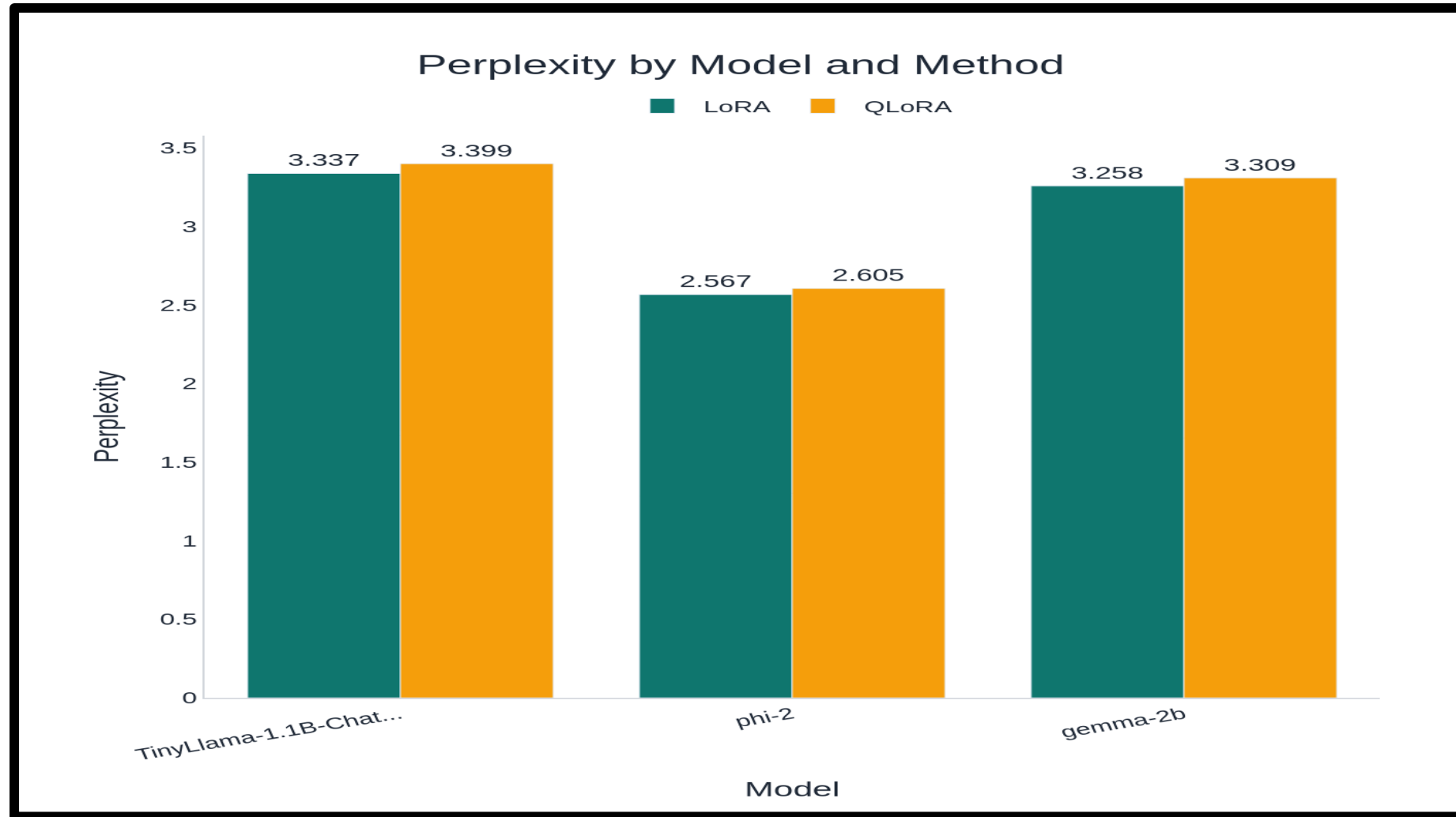
- **Tiny Llama & Phi-2: 32 heads vs Gemma-2B: 8** more parallel attention pathways for capturing contextual relationships.
- **Experiment impact:** More attention heads can increase attention computation, while Gemma's compact configuration can reduce attention overhead.



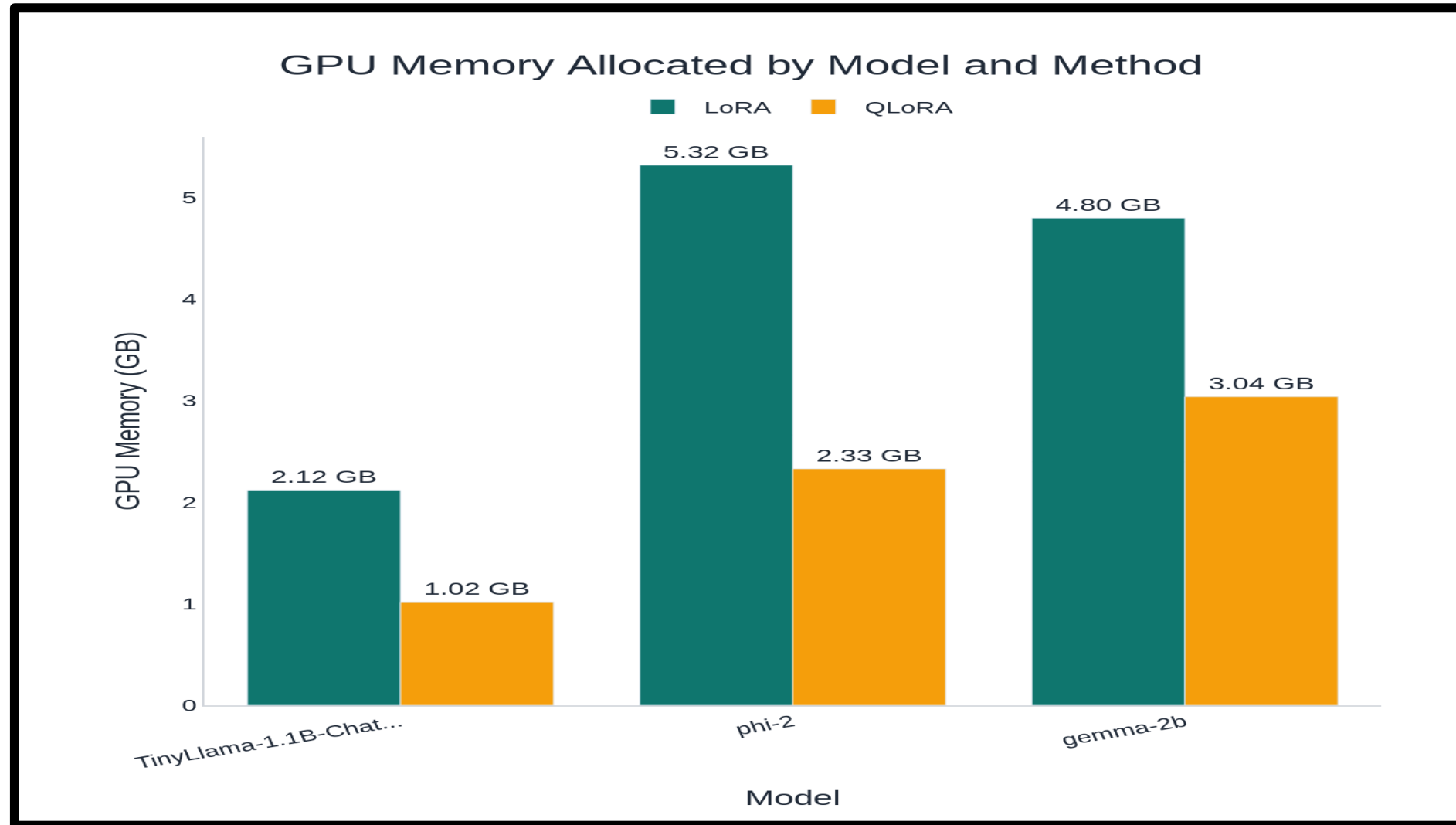
Vocabulary Size

- **Gemma-2B: 256K vs Phi-2: 51.2K and Tiny Llama: 32K** → substantially larger token vocabulary.
- **Experiment impact:** Larger vocabulary increases embedding/output-layer parameters and memory footprint, contributing to Gemma-2B's architectural overhead.

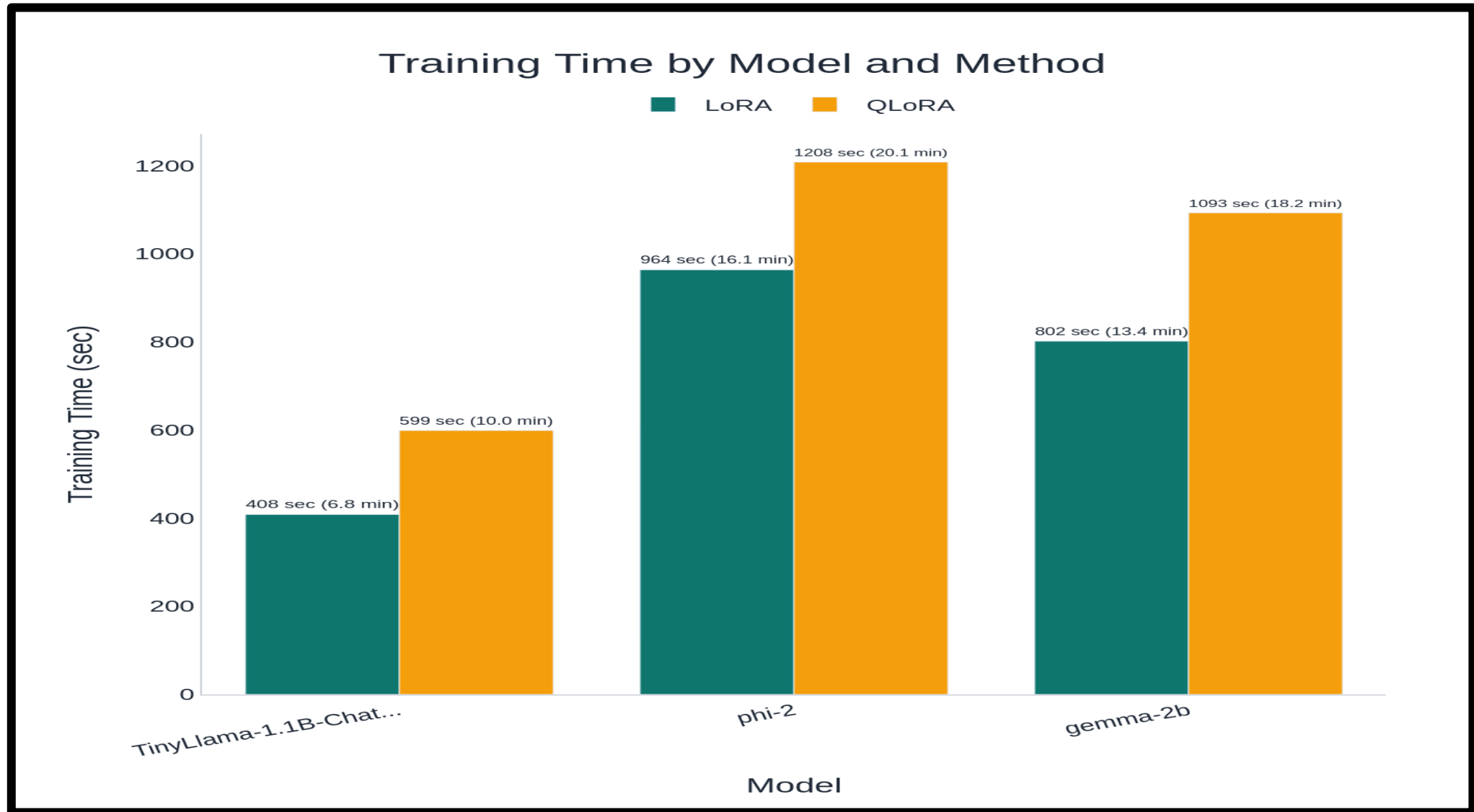
Model	Method	Training Samples	Epochs	Effective Batch Size	Learning Rate	Training Loss	Validation Loss	Perplexity	Training Time (sec)	GPU Memory Allocated (GB)
TinyLlama-1.1B-Chat-v1.0	LoRA	2000	1	16	0.0002	1.4507	1.2052	3.3375	408.3	2.12
TinyLlama-1.1B-Chat-v1.0	QLoRA	2000	1	16	0.0002	1.4729	1.2235	3.3992	598.98	1.02
Microsoft/phi-2	LoRA	2000	1	16	0.0002	1.3222	0.9427	2.5669	963.95	5.32
Microsoft/phi-2	QLoRA	2000	1	16	0.0002	1.3399	0.9576	2.6054	1208.3	2.33
Google/gemma-2b	LoRA	2000	1	16	0.0002	1.4588	1.181	3.2578	801.77	4.8
Google/gemma-2b	QLoRA	2000	1	16	0.0002	1.4806	1.1966	3.3087	1093.33	3.04



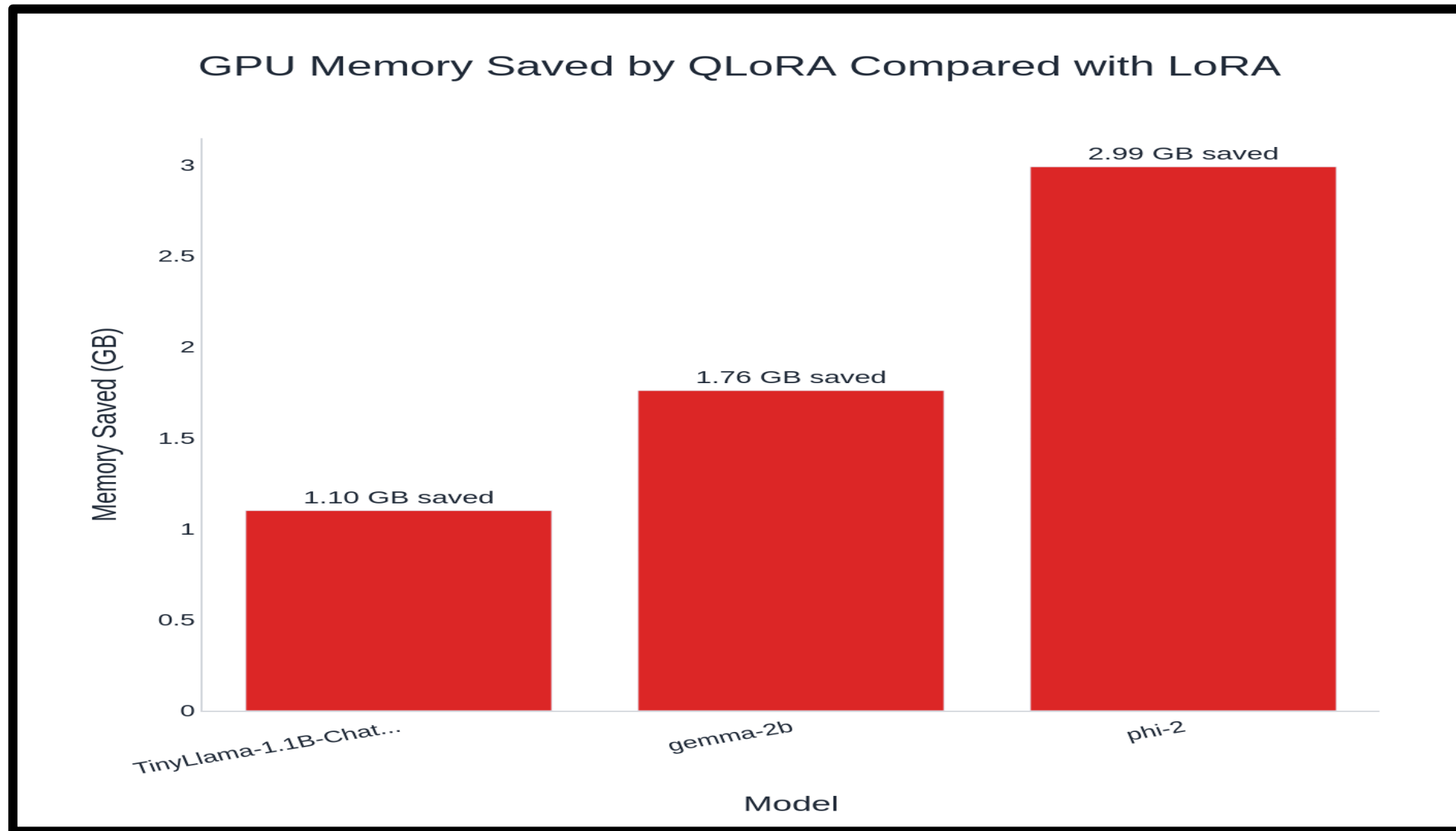
- Perplexity measures how well the LLM predicts the next token in a sequence.
- LoRA achieves slightly lower perplexity across all models, while QLoRA maintains comparable predictive performance with the advantage of substantially lower GPU-memory requirements.



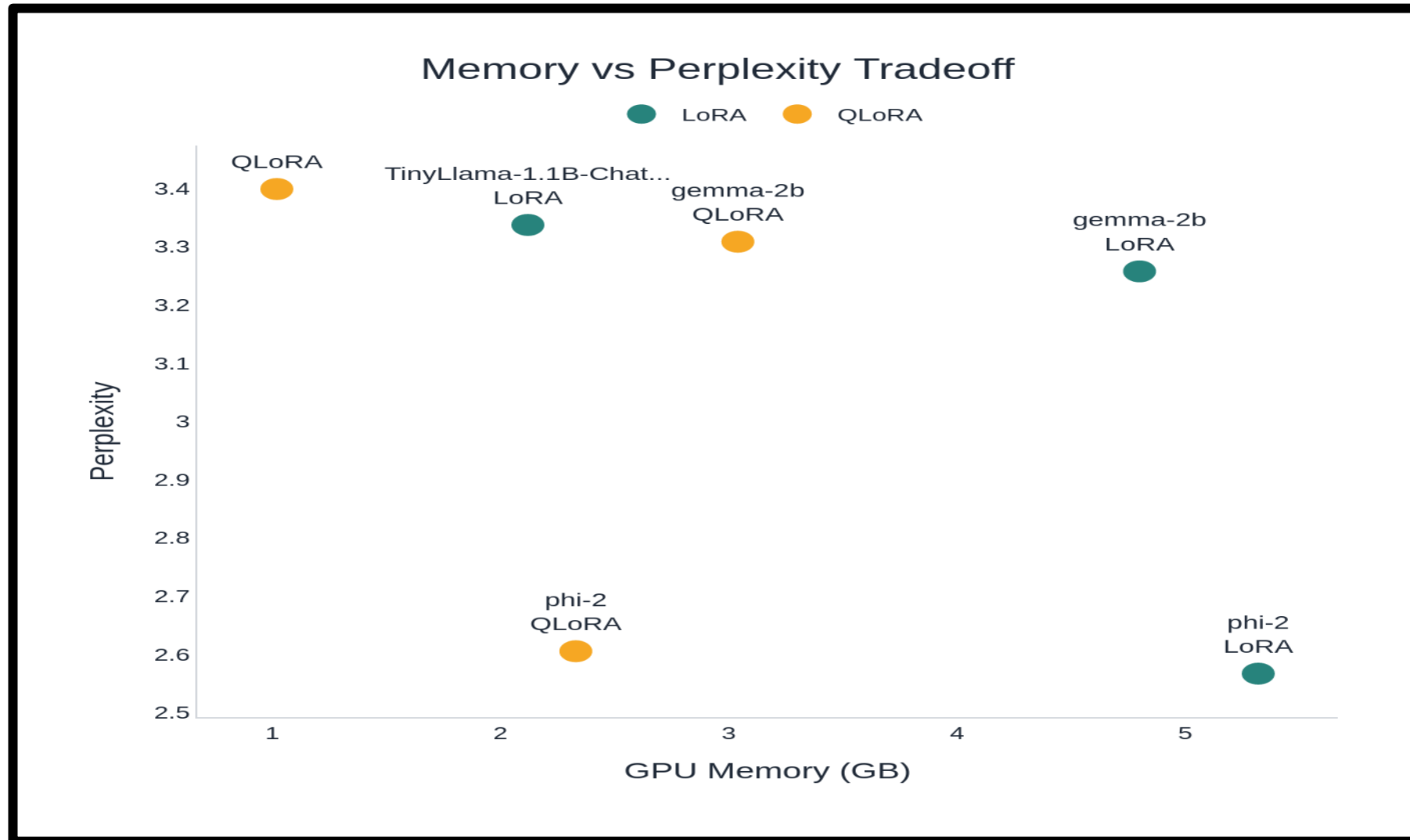
- QLoRA reduces GPU memory consumption by ~37–56% across the models while maintaining comparable fine-tuning performance, making larger LLMs more feasible on limited GPU hardware.



LoRA consistently reduces training time, while QLoRA requires more training time but significantly reduces GPU-memory requirements, making it suitable for memory-constrained hardware.



- QLoRA saves up to 2.99 GB (~56%) of GPU memory in our experiments, demonstrating its key advantage for memory-efficient LLM fine-tuning on constrained GPU hardware.



- QLoRA achieves substantial GPU-memory savings with only a small increase in perplexity, demonstrating an effective memory performance trade-off for resource-constrained LLM fine-tuning

Model-wise average qualitative scores

Model	Method	Fluency	Coherence	Relevance	Creativity	Story Quality
Gemma-2B	LoRA	5	4.33	5	5	4.83
Gemma-2B	QLoRA	5	5	5	4.33	4.83
Phi-2	LoRA	4.33	4.33	5	4.66	4.58
Phi-2	QLoRA	5	4.33	5	5	4.83
TinyLlama	LoRA	5	4.66	5	5	4.916
TinyLlama	QLoRA	5	4.66	5	5	4.916

Both LoRA and QLoRA produce high-quality, fluent and relevant stories; QLoRA maintains comparable generation quality while providing the memory-efficiency advantage demonstrated in the other experiments

- **Fluency:** Measures how smooth, natural and grammatically correct the generated story is.
- **Coherence:** Measures whether the story remains logically connected and consistent from beginning to end.
- **Relevance:** Measures how closely the generated story matches the given prompt/topic.
- **Creativity:** Measures the originality and variety of generated story content.
- **Story Quality:** Overall human-rated assessment combining the quality, coherence, relevance and fluency of the generated stories.

MODEL: TinyLlama-1.1B-Chat-v1.0

Prompt: Once upon a time, there was a little rabbit

****TinyLlama-1.1B-Chat-v1.0 (LoRA)****

Once upon a time, in a land far, far away, there was a little boy named Bob. He loved to play with his toys and do fun things. One day he found something special - it was a shiny new doll! It had big eyes and a friendly smile on its face.

Bob couldn't wait to show the doll to his mommy and daddy. He wanted them to be happy too, so he decided to take care of it very carefully.

But then, one day, Bob forgot all about his doll! He lost it when he ran around the house playing. The doll was gone forever, and Bob was sad.

Now, every time Bob sees a shiny new doll, he thinks of how important it is to keep treasures safe. He reminds himself that he should always look after what he has been given. And he hopes that someday his doll will come back home again, just like it did before.

****TinyLlama-1.1B-Chat-v1.0 (QLoRA)****

Once upon a time, in a land far, far away, there was a little boy named Bob. He loved to play with his toys and do fun things. One day he found a small box that had a red lid. He thought it might be something special so he opened the lid and inside there were many things.

Bob saw lots of balls, balloons, and other fun things. He started playing with them and had so much fun. Then he decided to put the box back on its lid, but he forgot! When he tried to put it back on, it wouldn't fit.

But then he remembered an important lesson - if you want to keep something safe, always make sure your lid is tightly shut. So now he kept his lid tightly closed when he played with his toys. And he continued to have lots of fun and play with all his new toys!

MODEL: Phi-2 (Microsoft/phi-2)

Prompt: Once upon a time, there was a little rabbit

****phi-2 (LoRA)****

Once upon a time, in a land far, far away, there was a little girl named Lily. She loved to play outside and explore the world around her. One day, she saw something very special on top of a big rock. It was a tiny kitten!

Lily climbed up the rock to get closer to the kitten. The kitten meowed at her and Lily knew that it needed help. So, she picked up the kitten and brought it back down to where she lived. Her mommy was so happy when she saw the kitten too. They gave the kitten lots of love and made sure it had everything it needed to be healthy and strong. And from then on, they all played together every day. The end.

****phi-2 (QLoRA)****

Once upon a time, in a land far, far away, there was a little girl named Lily. She loved to play outside and explore the world around her. One day, she saw something very big on top of a hill. It was a ladder!

Lily wanted to climb up the ladder so she could see what was at the top. But the ladder was too tall for her to reach. So, she decided to ask her friend Jack for help.

Jack came over and they tried to lift the ladder together. But it was still too heavy for them. Then, Lily had an idea. "Let's use your dad's truck!" she said excitedly.

They went to Jack's house and asked his dad if he could borrow the truck. His dad agreed, but only if they promised to be careful with it.

Lily and Jack loaded the ladder into the back of the truck and drove all the way to the top of the hill. ...

MODEL: Gemma-2B (Google)

Prompt: Once upon a time, there was a little rabbit

****gemma-2b (LoRA)****

Once upon a time, in a land far, far away, there was a little girl named Lily. She loved to play with her dolls and dress them up. One day, she saw a beautiful doll that was dressed in a yellow dress and blue shoes. It looked so cute! Lily wanted it for herself, but she didn't have enough money to buy it.

So, she decided to make her own yellow dress and blue shoes out of paper. She took some yellow and blue paper and cut them into different shapes. Then, she used glue to put them together and made a dress and shoes that looked just like the one she had seen.

Lily was so happy with her new dress and shoes. She wore them everywhere she went and showed them off to her friends. Everyone thought they were very pretty and said they wished they could be as creative as Lily.

The moral of the st...

****gemma-2b (QLoRA)****

Once upon a time, in a land far, far away, there was a little girl named Lily. She loved to play with her dolls and dress them up. One day, she had the idea to make a doll house for her dolls. She asked her mom if she could borrow some of her old clothes to use as decorations for the doll house. Her mom said yes and gave her some of her old shirts and dresses.

Lily started putting the clothes together to make a doll house. She took a box and cut it into two pieces. She put the pieces together to make a big box. Then, she took some more clothes and made a roof for the doll house. She used a piece of fabric to cover the box and make the roof.

Lily finished her doll house and was very proud of it. She decorated it with the clothes she had borrowed from her mom. She put her dolls inside the ...

LoRA

Pros

- Simple implementation without quantization.
- Only lightweight LoRA adapter matrices are trainable, reducing fine-tuning cost.
- Preserves full-precision pretrained weights, maintaining model quality.
- Well supported and easy to integrate using PEFT.

Cons

- Base model remains in FP16/FP32, requiring more GPU memory.
- Less suitable for very large models on memory-constrained hardware.
- Larger model memory footprint compared with QLoRA.

QLoRA

Pros

- Stores the pretrained model in 4-bit NF4, significantly reducing memory usage.
- Enables fine-tuning of larger language models on limited-memory GPUs.
- Maintains comparable performance while using a quantized base model.
- Retains the parameter efficiency of LoRA while improving memory efficiency.

Cons

- More complex implementation due to quantization and dequantization.
- May exhibit a slight reduction in accuracy depending on the quantization settings and task.
- Requires careful configuration of quantization parameters (e.g., NF4, compute dtype, double quantization).

Findings

- LoRA achieved slightly better language-modeling quality, with lower validation loss and perplexity across all three models.
- QLoRA reduced GPU memory substantially, up to **56.2%** in the experiments, while maintaining comparable quality.
- LoRA trained faster, while QLoRA required **25.4 - 46.7%** more training time, showing a clear memory–time trade-off.
- Story-generation quality was nearly equivalent: LoRA **4.83/5** vs QLoRA **4.75/5**.
- **Phi-2 + LoRA** achieved the best observed language-modeling performance, while **Tiny Llama + QLoRA** used the least GPU memory.

Practical Implications

- **LoRA:** Preferred when sufficient GPU memory is available and **training speed and maximum observed quality** are priorities.
- **QLoRA:** Preferred when **GPU memory is the main constraint**, enabling fine-tuning on more limited hardware.
- QLoRA provides a practical compromise: **substantial memory savings with only a small observed quality difference**.

LoRA provides the best observed quality and faster training, while QLoRA delivers substantial GPU-memory savings with only a small quality trade-off, making QLoRA particularly suitable for GPU-constrained fine-tuning

Future Work

- **Scale to Larger Models & Datasets:** Extend experiments from **2K/500 samples to 10K+ datasets** and larger LLMs to assess scalability and memory efficiency.
- **Systematic Ablation Studies:** Vary **LoRA rank/alpha/dropout and QLoRA training configurations** to quantify their individual impact on quality, memory and runtime.
- **Robust Evaluation:** Run **multiple seeds with mean $\pm$ standard deviation**, replace heuristic story scoring with **human/LLM-as-a-judge evaluation**, and perform comprehensive **peak GPU-memory profiling**.
- **Detailed Resource Profiling :** Analyze **peak GPU memory, runtime and throughput** across different configurations

Thank You